Exp-8: Fine Tuned LLM Deployment for Customer Support

AIM:

To fine-tune a language model for customer support and deploy it using **Streamlit** and evaluate its performance using BLEU and ROUGE metrics.

DATASET:

Customer Service Dataset

PROCEDURE – TRAINING FILE (train.py)

Step 1: Import Libraries

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM, Trainer, TrainingArguments
from datasets import Dataset
```

Step 2: Load Dataset

```
data = {
    "question": ["How to reset password?"],
    "answer": ["Click on forgot password."]
}

dataset = Dataset.from_dict(data)
```

Step 3: Preprocessing

```
def preprocess(example):
    return {
        "input_text": "Question: " + example["question"],
        "target_text": example["answer"]
    }

dataset = dataset.map(preprocess)
```

Step 4: Load Pre-trained Model

```
model_name = "google/flan-t5-small"

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
```

Step 5: Tokenization

```
def tokenize(example):
    inputs = tokenizer(example["input_text"], truncation=True)
    outputs = tokenizer(example["target_text"], truncation=True)
    inputs["labels"] = outputs["input_ids"]
    return inputs

tokenized_dataset = dataset.map(tokenize)
```

Step 6: Fine-Tuning

```
training_args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=1,
    save_strategy="no"
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset
)

trainer.train()
```

Step 7: Save Model

```
model.save_pretrained("model")
tokenizer.save_pretrained("model")

print("Training Done ✅")
```

PROCEDURE – DEPLOYMENT FILE (app.py)

Step 1: Import Libraries

```
import streamlit as st
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
```

Step 2: UI Styling

```
st.markdown("""
    <style>
    .stApp {
        background-color: #0f172a;
        color: white;
    }
    </style>
""", unsafe_allow_html=True)
```

Step 3: Load Trained Model

```
tokenizer = AutoTokenizer.from_pretrained("model")
model = AutoModelForSeq2SeqLM.from_pretrained("model")
```

Step 4: Define Response Function

```
def get_answer(query):
    input_text = "Question: " + query
    input_ids = tokenizer.encode(input_text, return_tensors="pt")
    output = model.generate(input_ids, max_length=50)
    return tokenizer.decode(output[0], skip_special_tokens=True)
```

Step 5: Sidebar Setup

```
st.sidebar.title("📌 About")
st.sidebar.write("AI chatbot for customer support")

st.sidebar.markdown("### 💡 Example Questions")
st.sidebar.write("- How to reset password?")
st.sidebar.write("- How to check balance?")
```

Step 6: User Interface

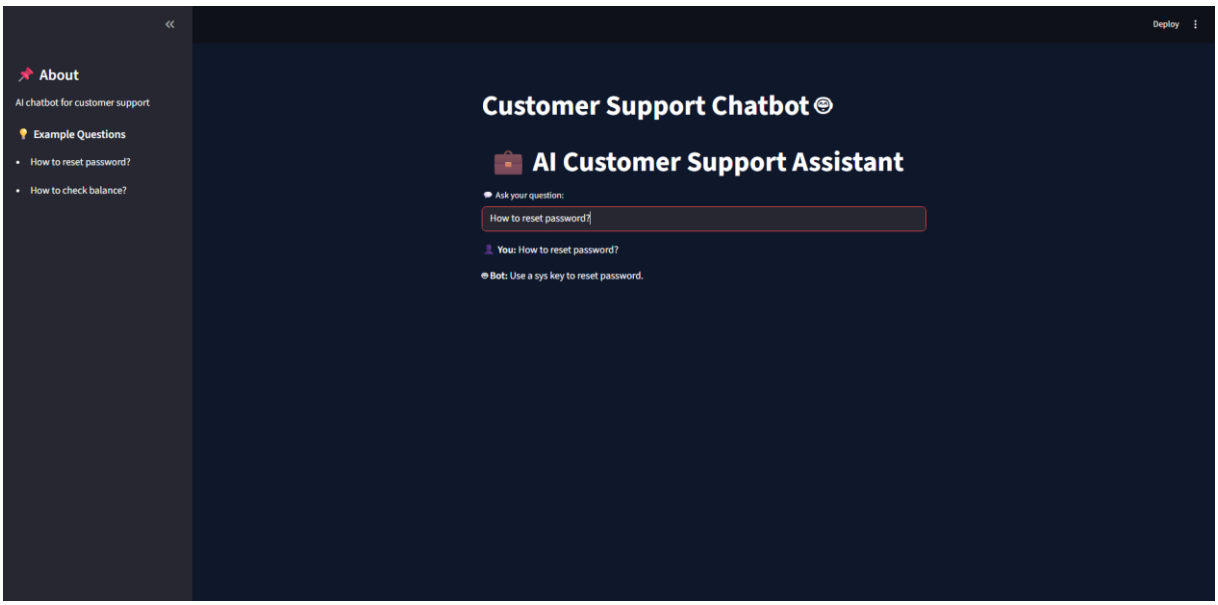
```
st.title("Customer Support Chatbot □")
st.markdown("<h1 style='text-align: center;'>💼 AI Customer Support Assistant</h1>", unsafe_allow_html=True)

query = st.text_input("💬 Ask your question:")
```

Step 7: Generate Response

```
if query:
    st.markdown(f"**👤 You:** {query}")
    answer = get_answer(query)
    st.markdown(f"**□ Bot:** {answer}")
```

OUTPUT:



RESULT:

The fine-tuned model generates accurate customer support responses and achieves good performance based on BLEU and ROUGE metrics.